16

O harness do Codex CLI

Os materiais anteriores construíram o Harness com Claude Code e OpenCode como referência. O Codex CLI da OpenAI segue a mesma filosofia — contexto estruturado, agents, hooks, MCP — mas com decisões de arquitetura próprias e um diferencial forte em **segurança e sandbox**. Este material mapeia cada peça do harness do Codex; o material 17 traz o comparativo lado a lado.

A IDEIA CENTRAL DESTE MATERIAL

O Codex trata configuração como política operacional do agent, não como preferência de usuário. Sandbox e aprovação são duas camadas independentes e explícitas — é o harness mais opinativo dos três em *o que o agent pode fazer*, não só em *o que ele deve saber*.

> **Nota de validade:** este material foi levantado a partir da documentação oficial do Codex em **agosto de 2026**. O produto evolui rápido — nomes de chaves, comandos e o status de recursos experimentais mudam entre versões. Confirme na sua versão antes de padronizar no time. Onde a informação não pôde ser confirmada na documentação oficial, está marcado no texto.

1 Arquivo de contexto: AGENTS.md

O equivalente ao CLAUDE.md é o **AGENTS.md** — e vale notar que é o mesmo nome usado pelo OpenCode, num movimento de convergência entre ferramentas.

Ordem de descoberta e mesclagem

1. **Global**: no home do Codex (`~/.codex`, ou o que `CODEX_HOME` apontar). Lê `AGENTS.override.md` se existir; senão, `AGENTS.md`.
2. **Projeto**: da raiz do repositório até o diretório atual, descendo pasta por pasta. Em cada nível verifica `AGENTS.override.md`, depois `AGENTS.md`, depois os nomes de fallback configurados. No máximo **um arquivo por diretório**.
3. **Mesclagem**: os arquivos são concatenados da raiz para baixo. Os mais próximos do diretório atual aparecem por último no prompt combinado — e por isso "vencem". Não é sobrescrita, é precedência textual.

> Dois detalhes que o Claude Code não tem: o `AGENTS.override.md`, que permite substituir a orientação herdada num diretório específico **sem apagar** o AGENTS.md base; e o limite explícito `project_doc_max_bytes` (padrão **32 KiB**) — ao atingir, o Codex simplesmente para de somar arquivos.

Há ainda `project_doc_fallback_filenames`, para times que já usam outro nome de arquivo (por exemplo `TEAM_GUIDE.md`). E o comando `/init` existe, gerando um scaffold do AGENTS.md para o projeto atual.

> ▸ **TRUQUE DE AUDITORIA**
>
> Rodando `codex -c log_dir=./.codex-log`, o arquivo `codex-tui.log` mostra **quais arquivos de instrução foram efetivamente carregados**. É a forma direta de descobrir por que o agent "ignorou" uma regra que você achava que estava no contexto.

2 Configuração: config.toml e profiles

Enquanto o Claude Code usa JSON e o OpenCode usa JSON/JSONC, o Codex usa **TOML**. A configuração vive em `~/.codex/config.toml` (usuário) e `.codex/config.toml` (projeto — carregado apenas se o projeto for marcado como *trusted*).

Precedência, do mais forte para o mais fraco

```
1. Flags de CLI e overrides com -c / --config
2. .codex/config.toml do projeto (raiz → cwd, o mais próximo vence)
3. Arquivo de profile selecionado com --profile
4. ~/.codex/config.toml            (usuário)
5. /etc/codex/config.toml          (sistema)
6. Defaults embutidos
```

Chaves principais

```
model = "<modelo>"
model_reasoning_effort = "minimal | low | medium | high | xhigh"
approval_policy = "untrusted" | "on-request" | "never"
sandbox_mode    = "read-only" | "workspace-write" | "danger-full-access"
web_search      = "cached" | "indexed" | "live" | "disabled"

[features]              # liga/desliga recursos (hooks, memories, multi_agent...)
hooks = true
```

O que é um "profile"

Um profile é um arquivo TOML separado, nomeado `~/.codex/<nome>.config.toml`, sobreposto ao config base quando você passa `--profile <nome>`. Serve para alternar posturas inteiras de trabalho com um único parâmetro:

```
# ~/.codex/deep-review.config.toml
model_reasoning_effort = "xhigh"
approval_policy = "on-request"
sandbox_mode = "read-only"

# uso:
codex --profile deep-review
```

> **Mudança recente:** em versões novas, o `--profile` deixou de ler blocos `[profiles.nome]` dentro do config.toml — os profiles agora vivem em arquivos dedicados. Se você encontrar tutoriais com a sintaxe antiga, ela está obsoleta.

Por segurança, a config de projeto **não pode** sobrescrever chaves ligadas a credenciais, provider, telemetria e notificações — o Codex ignora e avisa no startup. É uma proteção contra repositório malicioso que tenta redirecionar seu agent.

3 Skills

Skills no Codex seguem o mesmo formato conceitual do Claude Code: uma pasta com `SKILL.md` contendo `name` e `description` em frontmatter YAML, mais `scripts/`, `references/` e `assets/` opcionais.

> **Atenção ao caminho — é o erro mais comum:** as skills do Codex ficam em `.agents/skills/` (projeto) e `~/.agents/skills/` (usuário) — **não** em `.codex/skills/`. É o padrão aberto compartilhado com outras ferramentas. Muito tutorial de blog ainda repete o caminho antigo.

ESCOPO	CAMINHO
Repositório	`$CWD/.agents/skills` · `$REPO_ROOT/.agents/skills`
Usuário	`$HOME/.agents/skills`
Admin	`/etc/codex/skills`
Sistema	Empacotadas com o Codex

Como são invocadas

- **Explícita**: `/skills` abre o seletor; `$nome-do-skill` chama direto (repare: `$`, não `/`).
- **Implícita**: o Codex escolhe sozinho quando a tarefa casa com a `description` — o que reforça a regra do material 13: a description é o critério de decisão, escreva-a com cuidado.
- **Orçamento de contexto**: a lista inicial de skills ocupa no máximo 2% da janela de contexto ou 8.000 caracteres, o que for menor.

Há um `$skill-creator` embutido para gerar skills, e `$skill-installer <nome>` para instalar skills curadas. Para desabilitar uma skill sem apagá-la, use `[[skills.config]]` com `enabled = false` no config.toml.

> **Custom prompts foram descontinuados em favor de skills.** A página oficial de custom prompts traz literalmente o aviso de *deprecated*. Eles ainda funcionam (arquivos `.md` em `~/.codex/prompts/`, invocados como `/prompts:nome`, com `$1–$9` e `$ARGUMENTS`), mas o CLI exibe aviso de depreciação e sugere converter com o `$skill-creator`.

4 Subagents

Sim, o Codex tem subagents — e eles vêm **habilitados por padrão**. Três agentes embutidos: `default` (geral), `worker` (implementação) e `explorer` (exploração de código, leitura pesada).

Agents customizados são TOML, não Markdown

Aqui está a diferença estrutural mais visível frente a Claude Code e OpenCode, que usam Markdown com frontmatter. No Codex, cada agent é um arquivo **TOML autônomo**:

```
# .codex/agents/revisor.toml  (ou ~/.codex/agents/ para uso pessoal)
name = "revisor"
description = "Revisa código contra o AGENTS.md e a SPEC do projeto"
developer_instructions = """
Você é um engenheiro sênior especializado em code review.
Sua única responsabilidade é revisar — nunca modificar.
Classifique cada achado como BLOQUEANTE, IMPORTANTE ou SUGESTÃO.
"""

# aceita qualquer chave normal de config.toml:
model_reasoning_effort = "high"
sandbox_mode = "read-only"   ← o reviewer do material 10, travado por sandbox
```

Campos obrigatórios: `name`, `description` e `developer_instructions`. Um arquivo por agent. Se o nome colidir com um built-in (por exemplo `explorer`), o customizado sobrescreve.

> Repare no ganho prático: no Claude Code o "reviewer que não escreve" é garantido por *tools restritas + hook de bloqueio* (material 10). No Codex, você pode obter a mesma garantia declarando `sandbox_mode = "read-only"` no próprio agent — a restrição desce ao nível do sistema operacional.

Orquestração

A configuração global fica no bloco `[agents]`: `enabled`, `max_concurrent_threads_per_session`, `default_subagent_model`, `default_subagent_reasoning_effort`. Internamente há ferramentas de orquestração (`spawn_agent`, `send_input`, `wait_agent`, `close_agent`), mas a **delegação é explícita**: disparada por você ("delegue em paralelo") ou por instrução no AGENTS.md — não há delegação automática espontânea documentada. Subagents herdam a sandbox e a política de aprovação do turno pai, salvo sobrescrita própria.

Não existe recurso chamado "Agent Teams" no Codex; o equivalente prático são **workflows de subagents** especializados coordenados na mesma sessão.

5 Comandos e slash commands

Os nativos cobrem bastante terreno: `/init`, `/plan`, `/goal`, `/review`, `/compact`, `/model`, `/reasoning`, `/approve`, `/mcp`, `/hooks`, `/skills`, `/memories`, `/status`, `/worktree`, `/fork`, `/cloud`, `/side`, `/task`, entre outros. Skills habilitadas também aparecem na lista.

Para comandos personalizados, a orientação atual é **criar uma skill** — o mecanismo de custom prompts está descontinuado, como visto na seção 3. Um detalhe operacional: só arquivos `.md` no nível superior de `~/.codex/prompts/` são lidos (sem subpastas), e é preciso reiniciar o Codex após editar. Prompts personalizados por projeto (`.codex/prompts/`) **não são suportados** — existe apenas pedido aberto na comunidade.

6 Hooks

O sistema de hooks do Codex é robusto e cobre praticamente o mesmo ciclo de vida do Claude Code:

```
SessionStart · SessionEnd
SubagentStart · SubagentStop
PreToolUse · PostToolUse · PermissionRequest
PreCompact · PostCompact
UserPromptSubmit · Stop
```

Onde e como se configuram

Em `hooks.json` ou num bloco `[hooks]` dentro do config.toml, nos mesmos níveis: `~/.codex/` e `<repo>/.codex/`. A estrutura tem três níveis — evento → grupo com `matcher` (regex) → lista de handlers:

```
{ "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "hooks": [ { "type": "command",
                     "command": "./.codex/hooks/validate.sh",
                     "timeout": 30 } ] }
    ]
} }
```

É a mesma forma do Claude Code — quem já escreveu hooks lá porta o raciocínio direto. Hoje só o handler `type: "command"` é executado (`prompt` e `agent` são reconhecidos, mas ignorados).

Como bloqueiam

EVENTO	PODER DE DECISÃO
`PreToolUse`	`permissionDecision: "deny" | "allow"` — e pode reescrever a chamada com `updatedInput`
`PermissionRequest`	Aprova ou nega **antes** de a pergunta chegar a você
`PostToolUse`	`decision: "block"` substitui o resultado (não desfaz efeitos colaterais)
`Stop` / `SubagentStop`	`decision: "block"` força o agent a continuar — o gate de encerramento do material 12

O `exit 2` com mensagem em `stderr` também bloqueia, exatamente como no Claude Code.

Dois conceitos que o Codex tem e os outros não

- **Hooks precisam ser confiados.** Hooks não-gerenciados exigem revisão e aprovação (por hash de conteúdo) antes de rodar. O comando `/hooks` abre o navegador de hooks para inspecionar origem, revisar mudanças e desabilitar individualmente. É uma defesa direta contra repositório que traz hook malicioso.
- **Managed hooks.** Administradores definem hooks em `requirements.toml`; eles aparecem como *managed* e **não podem ser desabilitados** pelo usuário. Com `allow_managed_hooks_only = true`, todo hook de usuário/projeto/plugin é ignorado. É governança corporativa embutida.

Há ainda `"async": true` para rodar em background (até 8 simultâneos por sessão) — resolvendo nativamente o problema que no material 14 tratamos com `&` no shell. O `SessionEnd` sempre roda síncrono.

7 MCP

Registro por bloco no config.toml, com suporte a STDIO e HTTP:

```
# STDIO (processo local)
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

# HTTP remoto
[mcp_servers.figma]
url = "https://mcp.figma.com/mcp"
bearer_token_env_var = "FIGMA_OAUTH_TOKEN"
```

```
# pela CLI
codex mcp add <nome> --env VAR=VAL -- <comando> <args>
codex mcp list
codex mcp login <nome>      # OAuth
/mcp                        # status dos servidores conectados
```

Chaves úteis por servidor: `enabled`, `required`, `enabled_tools/disabled_tools`, `default_tools_approval_mode` (`auto | prompt | writes | approve`), `startup_timeout_sec` (10s) e `tool_timeout_sec` (60s). Repare no `default_tools_approval_mode`: é possível exigir aprovação apenas para as tools de escrita de um servidor — granularidade que os concorrentes não oferecem no mesmo nível.

8 Sandbox e aprovação: o diferencial do Codex

Este é o ponto onde o Codex se separa claramente dos outros dois. São **duas camadas independentes**:

CAMADA	PERGUNTA QUE RESPONDE	VALORES
Sandbox	O que o Codex *consegue* fazer tecnicamente	`read-only · workspace-write · danger-full-access`
Approval policy	Quando ele *precisa perguntar*	`untrusted · on-request · never`

A sandbox é reforçada pelo sistema operacional — Seatbelt no macOS, `bwrap` + `seccomp` no Linux, sandbox nativa no Windows. Não é uma promessa do modelo: é isolamento de processo.

```
# combinações mais usadas
Auto (padrão)          workspace-write + on-request
Leitura segura         read-only + on-request
CI somente leitura     read-only + never
Acesso total           --yolo  (sem sandbox, sem aprovação)

# testar o comportamento da sandbox localmente
codex sandbox linux -- <comando>
```

Permission Profiles

Camada mais granular (em beta à época deste material), que substitui o `sandbox_mode` — os dois não se combinam. Perfis embutidos: `:read-only`, `:workspace`, `:danger-full-access`. E perfis customizados em `[permissions.<nome>]`, com regras de **filesystem** (read/write/deny por path, incluindo globs) e de **rede** (allow/deny por domínio, com wildcards). O `extends` permite herdar de um perfil existente.

> É aqui que o Codex responde à pergunta do material 06 de um jeito diferente: em vez de um arquivo de exclusão que *pede* ao agent para não ler, ele nega no nível do sandbox — `"**/*.env" = "deny"` não é sugestão, é barreira.

Rules em Starlark

Uma camada extra de controle sobre comandos executados fora da sandbox, escrita em Starlark (`~/.codex/rules/default.rules`), com `prefix_rule(pattern=..., decision="allow"|"prompt"|"forbidden")` — testável com `codex execpolicy check`. Note que "Rules" no Codex significa **política de execução de comandos**, enquanto no Claude Code significa **instruções escopadas por path**. Mesmo nome, conceitos diferentes.

9 Exclusão de contexto: não existe .codexignore confiável

> Não há, na documentação oficial, um `.codexignore` funcional. Relatos da comunidade (issues do repositório) indicam que o arquivo não é respeitado de forma confiável. Na prática: o `.gitignore` e exclusões padrão cobrem parte do problema, e o mecanismo **oficial e reforçado** para impedir leitura de arquivos sensíveis são os **Permission Profiles** com regras `deny` — controle de segurança real, não filtro de contexto.

10 Planejamento: /plan e /goal

RECURSO	O QUE FAZ
`/plan`	Alterna o modo de planejamento multi-etapa na sessão. É o toggle simples, análogo ao Plan Mode do Claude Code.
`/goal`	Recurso mais robusto e sem equivalente direto nos concorrentes: define um **objetivo persistente** que o Codex persegue até concluir, pausar ou precisar de input. O texto do goal serve como prompt inicial *e* como critério de conclusão. Pode ser pausado, retomado, editado e limpo. Requer habilitar `features.goals`.

O fluxo recomendado combina os dois: usar `/plan` para moldar a abordagem e então fixar o objetivo com `/goal`. Existe também uma ferramenta interna de plano (`update_plan`), citada na documentação de hooks, análoga à lista de tarefas do Claude Code — mas sem página própria detalhando formato.

11 Memórias

O Codex tem memória persistente nativa — desligada por padrão, habilitada em `[features] memories = true`. O armazenamento é local, em `~/.codex/memories/`, com resumos, entradas duráveis e evidências das sessões anteriores. A geração roda em background, evita sessões curtas e **redige segredos automaticamente**. O comando `/memories` controla, por conversa, se ela pode usar e/ou gerar memórias.

> Em termos do material 14: o Codex entrega uma parte do RAG do projeto **pronta de fábrica**. Isso não elimina o RAG customizado — o banco próprio continua sendo mais controlável, versionável e compartilhável com o time — mas cobre bem o caso de uso individual.

12 Peças extras do harness

RECURSO	O QUE É
Plugins	Formato de *distribuição*: empacota skills + servidores MCP + assets num pacote instalável. Skills é o formato de autoria; plugin é como você compartilha.
/review	Modo dedicado de code review — revisa mudanças não commitadas ou compara contra uma branch base. Customizável por uma seção `## Code Review Rules` dentro do AGENTS.md.
Record & Replay	Grava um workflow feito manualmente e gera um rascunho de skill a partir da demonstração.
Worktrees	`/worktree` cria uma worktree Git isolada para a sessão; `/fork` copia o chat para outra sessão. Execução local, em worktree ou na nuvem.
codex exec	Modo não-interativo para CI. O Codex também pode ser embutido via SDK, exposto como App Server ou como servidor MCP, e usado como GitHub Action.
requirements.toml	Configuração corporativa: o admin força política de aprovação, sandbox, perfis permitidos e hooks obrigatórios — acima de tudo que o usuário configura localmente.
Feature Maturity	Todo recurso em `[features]` tem rótulo declarado: Experimental, Beta, Stable ou Deprecated. Consulte antes de padronizar algo no time.

▸ **COLA PARA FIXAR**

- Contexto: **AGENTS.md**, com `AGENTS.override.md` por diretório e limite de 32 KiB. Configuração em **TOML**.
- Skills vivem em **`.agents/skills/`** (não em .codex/) e se chamam com `$nome`. Custom prompts estão **descontinuados**.
- Subagents são arquivos **TOML** em `.codex/agents/` — e podem ter **sandbox própria**, travando o reviewer no nível do SO.
- Hooks cobrem o mesmo ciclo de vida, com dois extras: precisam ser **confiados** e podem ser **managed** pelo admin.
- O diferencial é **sandbox + approval como camadas independentes**, reforçadas pelo sistema operacional.
- Não existe `.codexignore` confiável: proteja arquivos sensíveis com **Permission Profiles** e regras `deny`.

ASA · Desenvolvimento Acelerado com IA — Material de apoio · Bloco E · 16 — O harness do Codex CLI